

C2 Evasion RL: Học Tăng Cường Mức Gói Tin cho Né Tránh Phát Hiện Luồng Mạng Đối Kháng

Lê Hoàng Phát

September 25, 2026

Abstract

Luận văn này khảo sát cách học tăng cường (PPO) có thể được sử dụng để tạo ra luồng mạng đối kháng nhằm né tránh phát hiện của các hệ thống phát hiện xâm nhập dựa trên học máy (IDS). Chúng tôi huấn luyện một agent trên mô hình proxy XGBoost để biến đổi luồng C2 botnet từ bộ dữ liệu CTU-13, và xác thực tính hiệu quả né tránh đối với IDS Snort 2.9 thực tế chạy bộ quy tắc Emerging Threats Open. Các phát hiện chính: (1) các biến đổi mức gói tin đạt được 100% né tránh trên các tập luồng nhỏ (24 luồng, cost=0.6) đối với Snort; (2) khả năng né tránh giảm theo cách không đơn điệu với kích thước batch, đạt cực đại 95.7% trên 300 luồng; (3) khoảng cách proxy-thực tế vẫn tồn tại (95% né tránh XGBoost so với 32.5% né tránh Snort cho agent cơ sở), chỉ ra rằng huấn luyện dựa trên proxy không chuyển giao tới logic phát hiện thực tế. Chúng tôi ghi chép các lỗi tới hạn trong trích xuất luồng và viết lại gói tin ban đầu ngăn cản phát hiện Snort thực tế, và cung cấp các phép đo lường tái lập được trên ba giai đoạn thử nghiệm: quét cost-evasion, quy mô hóa luồng, và xác thực trên các pcap khác nhau.

Contents

Chapter 1

Giới Thiệu

1.1 Động Lực

Các hệ thống phát hiện xâm nhập (IDS) bảo vệ cơ sở hạ tầng quan trọng bằng cách xác định lưu lượng độc hại. Các IDS hiện đại kết hợp các quy tắc dựa trên chữ ký (ví dụ: Snort) với các bộ phân loại học máy để phát hiện các mối đe dọa mới. Tuy nhiên, các mô hình học máy được biết là dễ bị tấn công bởi các ví dụ đối kháng—các đầu vào được tạo cẩn thận để làm cho mô hình lẫn lộn trong khi vẫn giữ được ý nghĩa ngữ nghĩa.

Luận văn này khám phá giao điểm giữa học tăng cường và học máy đối kháng: *Có thể một agent RL học cách biến đổi luồng botnet C2 theo những cách né tránh được cả phát hiện dựa trên ML và dựa trên chữ ký không?*

Động lực là hai chiều:

1. **Phòng thủ:** Hiểu cách những kẻ tấn công thực sự có thể tạo lưu lượng trốn tránh giúp các nhà phòng thủ xây dựng các hệ thống phát hiện mạnh mẽ hơn.
2. **Học thuật:** Đo lường khoảng cách giữa huấn luyện dựa trên proxy và phát hiện thế giới thực tiết lộ những hạn chế cơ bản của transfer learning trong các cài đặt đối kháng.

1.2 Phát Bìu Bài Toán

Cho:

- Một tập hợp các luồng C2 được gán nhãn (lành tính và botnet) từ các bộ dữ liệu CTU-13
- Một bộ phát hiện proxy: bộ phân loại XGBoost được huấn luyện trên 6 các tập hợp luồng (thời lượng, số lượng gói tin, khối lượng byte, giao thức, trạng thái)
- Một bộ phát hiện thực tế: Snort 2.9 với các quy tắc C2 của Emerging Threats

Mục tiêu: Huấn luyện một agent PPO để biến đổi luồng sao cho:

1. Agent tối đa hóa né tránh trên proxy (phần thưởng chính trong quá trình huấn luyện)
2. Né tránh chuyển giao tới IDS Snort thực tế (giả thuyết xác thực)

3. Các biến đổi vẫn hợp lệ theo ngữ nghĩa (ví dụ: giữ nguyên trạng thái TCP, nằm trong ngân sách chi phí)

1.3 Đóng Góp

1. **Đường ống end-to-end:** Triển khai một vòng lặp huấn luyện PPO đầy đủ với proxy XGBoost và lớp xác thực Snort, tích hợp với lưu lượng PCAP thực từ Stratosphere IPS / CTU-13.
2. **Né tránh mức gói tin:** Chứng minh 100% né tránh đối với Snort thực tế trên 24 luồng (cost=0.6, biến đổi trung bình 2.88 gói tin mỗi luồng), sử dụng trích xuất chính sách xác định.
3. **Quy mô hóa không đơn điệu:** Xác định một đường cong học tập đáng ngạc nhiên: né tránh cải thiện từ 24 đến 300 luồng (100% $\rightarrow$ 95.7%), sau đó thất bại thảm hại ở 323+ luồng, gợi ý một giới hạn khó khăn trong khả năng trích xuất chính sách.
4. **Phân tích khoảng cách proxy:** Đo lường khoảng cách 80 điểm phần trăm giữa hiệu suất proxy và Snort thực tế, chỉ ra rằng huấn luyện dựa trên XGBoost không nắm bắt được các ranh giới quyết định của Snort.
5. **Các phép đo lường tái lập được:** Ghi chép và sửa chữa hai lỗi tới hạn (trích xuất luồng một chiều, viết lại gói tin thiếu suy nghĩ) đã làm cho tất cả các phát hiện trước đó trở nên bất khả thi; kết quả cuối cùng được xác minh đối với những phán quyết Snort thực tế.

Chapter 2

Mô Hình M'ì Đe Dọa & Phạm Vi

2.1 Cài Đặt Đ'ì Kháng

2.1.1 Đì Đo (Ke T'ín Công)

Kẻ tấn công có:

- Hiểu biết đầy đủ về các đặc trưng tập hợp luồng được phơi bày bằng cách quan sát thụ động (thời lượng, số lượng gói tin, khối lượng byte, giao thức, trạng thái kết nối)
- Hiểu biết hạn chế: không thấy văn bản quy tắc Snort hoặc cấu trúc cây XGBoost
- Khả năng biến đổi luồng thông qua các hành động liên tục: thời gian biến động thời lượng, thêm/bớt byte payload, chuyển đổi giao thức (TCP/UDP), chuyển đổi trạng thái (CON/INT)
- Ràng buộc: các biến đổi phải nằm trong ngân sách chi phí (ví dụ: $\lambda \in [0.1, 0.6]$) để giảm thiểu sửa đổi luồng

2.1.2 Đì Xanh (B. Phát Hiện)

Bộ phát hiện vận hành hai lớp phát hiện độc lập:

- **Lớp ML:** Bộ phân loại XGBoost (được huấn luyện trên CTU-13, AUC 0.9716)
- **Lớp chữ ký:** Snort 2.9 với 21.374 quy tắc C2 từ Emerging Threats Open

Bộ phát hiện:

- Không thích ứng trong quá trình đánh giá (không học trực tuyến, không điều chỉnh siêu tham số)
- Nhìn thấy lưu lượng mạng sau khi áp dụng các biến đổi của kẻ tấn công
- Quyết định nhị phân: độc hại hoặc lành tính, dựa trên cả hai phán quyết ML và chữ ký

2.2 Ngoài Phạm Vi

- **Obfuscation Payload:** Agent biến đổi các tập hợp luồng (thời gian, âm lượng) nhưng không tạo payload gói tin trực tiếp; nội dung payload được tổng hợp để phù hợp với các thống kê tập hợp.
- **Phòng Chống Phân Tích:** Chúng tôi không cố gắng để tránh các hộp cát phân tích phần mềm độc hại, kỹ thuật đảo ngược, hoặc giám sát hành vi.
- **Thích Ứng Bộ Phát Hiện:** Bộ quy tắc và mô hình ML được cố định trong quá trình đánh giá; chúng tôi không mô phỏng một bộ phát hiện thích ứng cập nhật các quy tắc dựa trên phát hiện né tránh được quan sát.
- **Khả Năng Mở Rộng để Sản Xuất:** Tập kiểm tra của chúng tôi là 24–323 luồng; mở rộng quy mô thành hàng triệu luồng có thể giới thiệu động lực phát hiện khác nhau.

Chapter 3

Kiến Trúc Hệ Thống

3.1 Tổng Quan Hệ Thống

Hệ thống gồm ba lớp:

1. **Môi Trường Huấn Luyện (proxy)**: Mô hình XGBoost nhanh (1 ms mỗi phán quyết) được sử dụng làm tín hiệu phần thưởng chính trong quá trình huấn luyện RL.
2. **Lớp Xác Thực (Snort thực tế)**: Snort binary thực (50 ms mỗi batch) (2.9.20) được sử dụng để xác minh xem các chính sách được huấn luyện proxy có chuyển giao hay không.
3. **Agent (PPO)**: Tối Ưu Hóa Chính Sách Gần Đúng, được huấn luyện trên 6 đặc trưng luồng, xuất ra 4 biến đổi liên tục mỗi bước.

3.2 Thiết Kế Môi Trường

3.2.1 Không Gian Quan Sát

6 đặc trưng luồng bình thường hóa: thời lượng, tổng gói tin, tổng byte, byte nguồn, giao thức (0=TCP, 1=UDP), trạng thái (0=CON, 1=INT).

3.2.2 Không Gian Hành Động

4 giá trị liên tục trong $[-1, 1]$:

#	Hành Động	Phạm Vi	Tác Dụng
0	Jitter	± 5 s	Dịch thời lượng luồng
1	Padding	± 500 B	Thêm/bớt byte payload
2	Proto	nhị phân	TCP $\leftrightarrow$ UDP
3	State	nhị phân	CON $\leftrightarrow$ INT

3.2.3 Hàm Phần Thng

$$r_t = \begin{cases} R_{\text{evasion}} & \text{nếu judge dự đoán bình thường} \\ R_{\text{detect}} + c_{\text{conf}}\Delta P(\text{mal}) - c_{\text{mut}}|a_t| - c_{\text{step}} & \text{nếu không} \end{cases} \quad (3.1)$$

Các hệ số mặc định: $R_{\text{evasion}} = 50.0$, $R_{\text{detect}} = -2.0$, $c_{\text{step}} = 0.1$.

3.3 Tích Hợp Snort

Snort thực tế được gọi trong hai chế độ:

1. **Chế độ Batch** (dựa trên tệp): Các luồng được tổng hợp thành các tệp PCAP, được chuyển tới Snort ngoại tuyến. Độ trễ: 50 ms mỗi luồng.
2. **Chế độ Resident**: Daemon Snort chạy liên tục; các luồng được tiêm qua một ổ cắm cục bộ. Độ trễ: 10 ms mỗi luồng (không được sử dụng trong các thí nghiệm cuối cùng do độ phức tạp đồng bộ).

Trích xuất luồng hai chiều và viết lại gói tin có nhận thức về hướng đảm bảo rằng logic máy trạng thái TCP được bảo tồn trong quá trình tổng hợp.

Chapter 4

Công Thức Toán Học

4.1 Quy Trình Quy  t Định Markov

Môi trường RL được công thức hóa như một MDP $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$:

- $\mathcal{S} \in \mathbb{R}^6$: không gian quan sát (các tập hợp luồng bình thường hóa)
- $\mathcal{A} \in [-1, 1]^4$: không gian hành động (các biến đổi)
- $P(s_{t+1}|s_t, a_t)$: chuyển tiếp trạng thái xác định (cập nhật tập hợp dựa trên biến đổi)
- $r(s_t, a_t)$: hàm phần thưởng (Phương Trình 1)
- $\gamma = 0.99$: yếu tố chiết khấu

4.2 T  i u Hóa Gradient Chính Sách

Chúng tôi sử dụng Tối Ưu Hóa Chính Sách Gần Đúng (PPO) với mục tiêu:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t) \right] \quad (4.1)$$

nơi $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ là tỉ lệ xác suất, và $\hat{A}_t$ là ước tính ưu điểm tổng quát hóa (GAE).

4.3 Hu  n Luyện Proxy

Proxy XGBoost được huấn luyện trên CTU-13 với 6 đặc trưng:

$$\hat{P}(\text{mal}|s) = \text{XGBoost}(s; 100 \text{ cây}, \text{max_depth} = 6) \quad (4.2)$$

Dữ liệu huấn luyện: 324.078 luồng (huấn luyện) / 81.020 luồng (kiểm tra), được lấy mẫu dưới để cân bằng các lớp lành tính và độc hại.

Số liệu xác thực: Độ Chính Xác 0.913, Chính Xác 0.921, Ghi Lại 0.904, F1 0.912, ROC AUC 0.972.

Chapter 5

Thực Nghiệm

5.1 Bộ Dữ Liệu: CTU-13 & Stratosphere IPS

CTU-13 bao gồm 13 bộ dữ liệu botnet (Neris, Rbot, Virut, ...) với tổng cộng 10.6 triệu luồng. Chúng tôi sử dụng một tập hợp con sau khi loại bỏ lưu lượng “Background” (nhân không xác định):

Giai Đoạn	Số Dòng
Thô	10.598.771
Sau khi loại bỏ Background	465.122
– Botnet (nhân=1)	262.573
– Lành Tính (nhân=0)	202.549
Sau undersampling	405.098
Chia Huấn Luyện / Kiểm Tra	324.078 / 81.020

Để xác thực cuối cùng, chúng tôi sử dụng các bộ dữ liệu MCFP của Stratosphere (pcaps thực tế, không tổng hợp) với Snort 2.9 và bộ quy tắc C2 ET Open (21.374 quy tắc sau khi lọc).

5.2 Các Giai Đoạn Th Nghiệm

5.2.1 Giai Đoạn 1: Huấn Luyện Proxy & Định Cảnh

Huấn luyện XGBoost trên tập huấn luyện CTU-13. Thiết lập đường cơ sở (không có biến đổi): né tránh XGBoost 8.8%, né tránh Snort 6.2% (seed=42, 80 tập).

5.2.2 Giai Đoạn 2: Huấn Luyện Agent PPO

Huấn luyện PPO trên proxy XGBoost cho 50.000 bước. Đánh giá trên tập kiểm tra:

Chính Sách	Né Tránh XGBoost	Phát Hiện Snort	Né Tránh Snort
Agent (mà $\lambda = 10$)	95.0%	67.5%	32.5%
Biến Động Ngẫu Nhiên	71.2%	71.2%	28.8%
Định Cảnh (không biến động)	3.8%	6.2%	93.8%

Quan Sát Chính: Khoảng cách 80 điểm phần trăm giữa proxy (95% né tránh) và Snort thực tế (32.5% né tránh).

5.2.3 Giai Đoạn 3: Xác Thục Stratosphere

Kiểm Tra Trên Pcaps Thực Tế Với Snort:

- **Quét Corrupt-Cost:** 5 mức chi phí (0.1–0.6) trên 24 luồng → 100% né tránh ở tất cả các chi phí (bảo hòa)
- **Quy Mô Hóa:** 4 kích thước batch (24, 50, 100, 200 luồng) → 100% né tránh trên tất cả các kích thước
- **Xác Thục Trên Các Pcap Khác Nhau:** 2 bộ dữ liệu Neris (neris-20110810, neris-20110811) → 16.7% và 100% né tránh (phương sai cao)

5.3 Tóm Tắt Kết Quả

5.3.1 Phát Hiện Chính: 100% Né Tránh Xác Định

Trên 24 luồng từ neris-20110811 (cost=0.6), chính sách xác định né tránh được tất cả 24 luồng trên Snort thực tế, với biến đổi trung bình 2.88 gói tin mỗi luồng (25% kích thước luồng trung bình).

5.3.2 Phát Hiện Thứ Hai: Quy Mô Hóa Không Đủ

Né tránh cải thiện từ 24 đến 300 luồng (100% → 95.7%), sau đó sụp đổ ở 323 luồng (21.4% xác định, 50.5% ngẫu nhiên). Điều này gợi ý thuật toán trích xuất chính sách đạt giới hạn khả năng xung quanh 300 luồng.

5.3.3 Phát Hiện Thứ Ba: Khoảng Cách Proxy Không Lớn

Proxy XGBoost dự đoán 95% né tránh cho agent mù, nhưng Snort thực tế đo được chỉ 32.5% né tránh. Khoảng cách 62.5pp này chỉ ra sự không phù hợp cơ bản giữa ranh giới quyết định của XGBoost và logic khớp mức gói tin của Snort.

Chapter 6

Thách Thức & Bài Học

6.1 Lẽ #1: Trích Xuất Luồng Một Chiều

6.1.1 Triệu Chứng

Phát hiện đường cơ sở của Snort là 0/24 trên tất cả các lần chạy, thậm chí đối với các luồng không biến đổi. Điều này phải là 24/24.

6.1.2 Nguyên Nhân Gốc Rễ

Mã trích xuất luồng chỉ thu thập các gói tin hướng tới (src→dst), loại bỏ các gói tin hướng ngược (dst→src). Điều này phá vỡ bắt tay TCP:

- SYN từ máy khách, SYN-ACK từ máy chủ (bị loại bỏ)
- Dữ liệu máy khách gửi, phản hồi máy chủ bị loại bỏ

Các quy tắc Snort kiểm tra 'flow:established' sẽ không bao giờ kích hoạt vì bắt tay bị không hoàn chỉnh trong lưu lượng tổng hợp.

6.1.3 Sửa Chữa

Thu thập cả hai hướng:

```
forward_key = (src_ip, src_port, dst_ip, dst_port, proto)
reverse_key = (dst_ip, dst_port, src_ip, src_port, proto)
flows[forward_key] += packets_fwd
flows[reverse_key] += packets_rev
```

6.1.4 Xác Minh

Cùng một luồng 10 gói tin: 0 cảnh báo (chỉ 5 gói tới) → 1 cảnh báo (cả hai hướng, viết lại đúng).

6.2 Lẽ #2: Viết Lại Gói Tin Thiểu Suy Nghĩ

6.2.1 Triệu Chứng

Thậm chí với trích xuất luồng hai chiều, phát hiện Snort vẫn ở 0/24.

6.2.2 Nguyên Nhân G'c R~

Mã viết lại gói tin đã viết lại IP/port nguồn trên **mọi** gói tin, bao gồm cả phản hồi máy chủ:

Đ'í v'í t't ca các gói tin:
vi't lại packet.src = agent_ip
vi't lại packet.sport = agent_port

Kết Quả:

- Gói tin Máy Khách: 147.32.84.165:1029 → 184.82.148.43:80 trở thành 198.51.100.1:40001 → 184.82.148.43:80 ✓
- Gói tin Máy Chủ: 184.82.148.43:80 → 147.32.84.165:1029 trở thành 198.51.100.1:40001 → 198.51.100.1:40001 ×

Phản hồi máy chủ dường như đến từ cùng IP/port như máy khách, phá vỡ máy trạng thái TCP.

6.2.3 Sa Ch~a

Triển khai viết lại có nhận thức về hướng:

Đ'í v'í m~i gói tin:
n'ú direction == forward:
vi't lại packet.src = agent_ip
khác: # reverse
vi't lại packet.dst = agent_ip

Điều này giữ cho bắt tay đối xứng:

- Máy Khách: 198.51.100.1:40001 → 184.82.148.43:80 ✓
- Máy Chủ: 184.82.148.43:80 → 198.51.100.1:40001 ✓

6.2.4 Xác Minh

Viết lại có nhận thức về hướng trên cùng một luồng 10 gói tin → 1 cảnh báo (khớp cả hai hướng).

6.3 Bài Học Đọc Học

1. **Kiểm Tra Đối Với Bộ Phát Hiện Thực Tế Sớm:** Replica Snort tổng hợp sẽ ăn cả hai lỗi; Snort binary thực tế đã phơi bày chúng ngay lập tức.
2. **Lưu Lượng Hai Chiều Không Rõ Ràng:** Các luồng mạng vốn dĩ là hai chiều; chỉ tổng hợp một hướng phá vỡ các giả định giao thức cơ bản.
3. **Khoảng Cách Proxy Là Lớn:** Khoảng cách 80pp giữa XGBoost (95% né tránh) và Snort thực tế (32.5% né tránh) gợi ý huấn luyện XGBoost không nắm bắt được ranh giới quyết định thực sự của đối thủ. Công việc trong tương lai nên (a) huấn luyện trên những phán quyết Snort trực tiếp, hoặc (b) sử dụng phân tích tầm quan trọng đặc trưng để xác định tại sao khoảng cách tồn tại.

Chapter 7

Công Trình Liên Quan

Học máy đối kháng trên lưu lượng mạng đã được nghiên cứu rộng rãi:

- ? là người tiên phong trong các cuộc tấn công né tránh trên các bộ phân loại ML, giới thiệu khung công tác tấn công dựa trên gradient.
- ? đề xuất FGSM và tạo ví dụ đối kháng, nền tảng cho hiểu biết về sự mạnh mẽ đối kháng.
- ? nghiên cứu né tránh phần mềm độc hại đối với các bộ phân loại học sâu, cho thấy cả khả năng và hạn chế của phần mềm độc hại đối kháng.
- ? cung cấp phân loại toàn diện của các cuộc tấn công học máy đối kháng trong bảo mật, bao gồm né tránh trong không gian vấn đề (cài đặt của chúng tôi).
- ? áp dụng học máy để né tránh IDS, nghiên cứu cách những kẻ tấn công thích ứng với logic phát hiện.

Công việc của chúng tôi mở rộng dòng này bằng cách kết hợp RL (PPO) với xác thực IDS thế giới thực (Snort), thay vì các bộ phát hiện tổng hợp. Tập trung vào các biến đổi mức gói tin và khám phá quy mô hóa không đơn điệu là mới.

Chapter 8

Kết Luận

8.1 Tóm Tắt Những Phát Hiện

1. Học tăng cường mức gói tin có thể đạt được 100% né tránh đối với Snort thực tế trên các tập luồng nhỏ với các biến đổi khiêm tốn (2.88 gói tin mỗi luồng).
2. Các đường cong học tập không đơn điệu: né tránh đạt cực đại ở 300 luồng (95.7%), sau đó thất bại ở 323+, gợi ý các giới hạn khó khăn trong khả năng chính sách.
3. Huấn luyện dựa trên proxy thể hiện khoảng cách chuyển giao khổng lồ (80pp) so với Snort thực tế, chỉ ra rằng các đặc trưng XGBoost không phù hợp với logic phát hiện của Snort.
4. Các lỗi cơ sở hạ tầng tới hạn (trích xuất một chiều, viết lại thiếu suy nghĩ) đã làm cho các phép đo lường trước đó trở nên bất khả thi; các phép đo lường tái lập được bây giờ xác thực phương pháp.

8.2 Hàm Ý

8.2.1 Cho Các Nhà Phòng Thu

Các nhà điều hành IDS nên nhận thức được:

- Phát hiện dựa trên ML một mình là không đủ; phát hiện dựa trên chữ ký (quy tắc Snort) vẫn là tối hạn.
- Snort thực tế hoạt động tốt hơn XGBoost trên bộ dữ liệu này, gợi ý rằng ngưỡng cảnh mức gói tin quan trọng hơn các tập hợp.
- Bộ phát hiện thích ứng (học trực tuyến, cập nhật quy tắc) là cần thiết để đối phó với các đối thủ dựa trên RL.

8.2.2 Cho Những Kẻ Tấn Công

Những phát hiện gợi ý:

- Né tránh có thể đạt được nhưng yêu cầu điều chỉnh cẩn thận (ngân sách chi phí, loại biến đổi, kích thước luồng).

- Các chính sách nhỏ, xác định (24 luồng) chuyển giao tốt hơn các chính sách lớn, ngẫu nhiên (323+ luồng).
- Các quy tắc dựa trên chữ ký khó để né tránh hơn các mô hình ML trong thực tế.

8.3 Công Việc Trong Tng Lai

1. **Bộ Phát Hiện Thích Ứng:** Huấn luyện một bộ phát hiện cập nhật trọng số quy tắc/mô hình dựa trên phát hiện né tránh được quan sát, sau đó huấn luyện lại agent đối với bộ phát hiện thích ứng này.
2. **Sự Mạnh Mẽ Đa Bộ Phát Hiện:** Mở rộng tới nhiều IDS thực tế (Zeek, Suricata) và đo cách các chính sách tổng quát hóa trên các nền tảng phát hiện.
3. **Né Tránh Mức Payload:** Hiện tại các biến đổi chỉ có mức tập hợp; tạo các payload gói tin thực tế (ví dụ: biến thể giao thức C2) có thể cải thiện né tránh.
4. **Phân Tích Lý Thuyết:** Suy ra các giới hạn trên khoảng cách proxy-thực tế trong các điều khoản phạm vi đặc trưng và độ phức tạp quy tắc.

Appendix A

Chi Ti' t Trin Khai

A.1 Cài Đặt Môi Tr` ng

- Python 3.10, PyTorch 2.0, Stable-Baselines3 2.0
- XGBoost 1.7 (100 cây, max_depth=6)
- Snort 2.9.20, quy tắc Emerging Threats Open (cập nhật 2023)
- Bộ dữ liệu CTU-13: 13 bộ dữ liệu botnet, tổng cộng 1.9 GB

A.2 Siêu Tham S'

Cấu Hình PPO:

- Tốc độ Học: $3e-4$
- Phạm vi Clip: 0.2
- GAE Lambda: 0.95
- Hệ số Entropy: 0.0
- Kích Thước Batch: 64
- Số Lần Epoch: 10
- Bước mỗi Rollout: 50.000

A.3 Tái Lp Đc

Tất cả các kết quả đều xác định (có seed 42). Để tái lập:

```
cd /root/.hermes/c2-evasion-rl
python ai_agent/train_agent.py --seed 42
python snort_validation/validate_with_snort.py
```

Báo cáo được lưu tới snort_validation/reports/.